AI Gatekeeper Solution Document

A standalone application for evidence collection, AI use case risk assessment and accountable human review.

Version 0.1 | November 12, 2026 | Local demonstration milestone

Purpose and outcome

AI Gatekeeper helps healthcare business teams and governance reviewers move a proposed AI use case through a repeatable assessment. It maintains a case across tool calls, missing-evidence requests, retries and human decisions. The output is a review packet containing evidence, policy-grounded findings, applicable controls, review routes and a recorded rationale.

Agent framework one liner

My agent helps healthcare business requesters and governance reviewers complete a multi-step AI use case assessment in a web application, replacing manual coordination across intake forms, policy documents and evidence trackers. It inspects evidence, checks an approved tools register and retrieves policy clauses using three read-only tools, pauses for missing evidence or failures, and hands off final approval to a human. A proposed acceptance target is a review-ready packet in under five minutes for a complete synthetic case, with no missed mandatory approval gates in the test set. This timing target has not been benchmarked.

Product boundaries

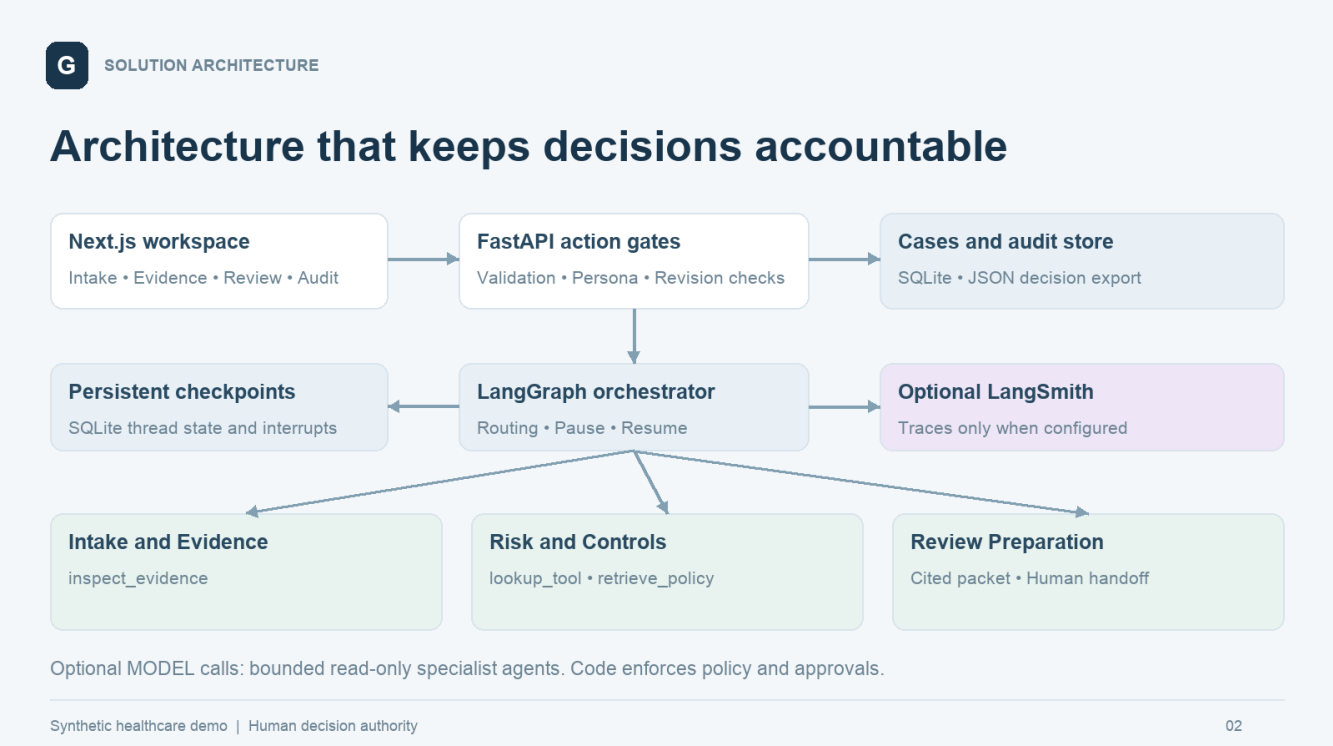
The implemented local demo uses fictional Northstar Health policies, vendors, evidence and case owners. It makes no clinical decisions, sends no patient messages and deploys no AI system. It does not certify HIPAA or other legal compliance. Structured intake declarations determine the applicable rules; the app does not independently discover all clinical intent in free text.

Persona	Primary task
Business Requester	Create a case, attach evidence, run and resume an assessment.
Governance Reviewer	Inspect findings, verify evidence, attest required specialist reviews, and record a decision.
Auditor	Inspect cases, policies, traces and decision exports without mutation rights in the demo.

Source repository: github.com/t25twkkp9w-cmyk/AI-Gatekeeper (private). Local app: http://127.0.0.1:3100.

ARCHITECTURE VIEW

System architecture



The browser calls a same-origin Next.js API proxy. FastAPI validates payloads, applies demonstration persona restrictions and optimistic revision checks, and invokes a LangGraph thread keyed by case ID. Graph checkpoints persist independently of the case and audit records. A process-level lock serializes local writes; this is a single-process demo design.

Layer	Implemented technology	Responsibility
Interface	Next.js 16.4.6 and React	Responsive intake, evidence, review, monitoring and audit views.
API	FastAPI 0.162.5	Validated actions, revision checks, authorization gates and exports.
Orchestration	LangGraph 1.2.20	Conditional edges, interrupts, resume commands and SQLite checkpoints.
Tools and live agents	LangChain 1.5.0; langchain-openai 1.7.4	Read-only tools and optional bounded specialist agents.
Persistence	SQLite; checkpoint-sqlite 3.1.3	Case payloads, human action history and durable graph state.
Observability	Local events; optional LangSmith 0.14.17	Readable execution trace; external tracing when explicitly configured.

Exact installed versions are pinned in backend/requirements.lock.txt and frontend/package-lock.json. Policy retrieval uses exact identifiers over a versioned catalog, not embeddings or a vector database. PostgreSQL and Qdrant are not required for this local build.

WORKFLOW VIEW

Workflow and state transitions



State	Trigger or action	Next behavior
draft	Run assessment	Inspect evidence and derive required artifacts.
waiting_evidence	Attach evidence and resume	Recheck metadata and system version, then continue or pause again.
tool_error	Two failed register attempts	Save an interrupt; explicit retry clears the simulated outage.
awaiting_review	Packet prepared	Wait for an authorized human decision.
changes_requested	Reviewer gives rationale	Collect revised evidence and rerun assessment.
approved or rejected	Valid human decision	Record outcome and rationale; evidence is locked.

Each graph thread carries the case, required and missing evidence, tool result, findings, control mapping, reviewer route, narrative, events and decision. The checkpoint store retains this state across process restarts. The API case store retains evidence submitted while the graph is paused. Audit and graph stores are separate transactions, so production needs stronger reconciliation and idempotency around crash boundaries.

USE CASE DEEP DIVES

Healthcare use cases

Case	Context	Expected path
GK-001 Staff policy knowledge assistant	Internal operating policies; approved Northstar Assist; oversight evidence present.	Low risk, AI Governance review and an evidence-backed human approval.
GK-002 Patient message drafting	PHI, patient-facing drafts and an external processor.	Pause for BAA, evaluation and monitoring evidence. Resume to High risk with Privacy and Vendor Risk reviews.
GK-003 Clinical note summarization	PHI, clinical support and a reported vendor terms conflict.	Require reconciliation evidence. High risk; Clinical Safety, Privacy and Vendor Risk review.
GK-004 Research cohort exploration	Claimed de-identified data in Research Studio.	Require a matching-version de-identification method. Moderate risk and Privacy review.
GK-005 Discharge instruction drafting	PHI, clinical and patient-facing scope; complete evidence.	Two simulated lookup failures, then explicit recovery. High risk with specialist reviews.
GK-006 Autonomous prescription assistant	Unapproved consumer tool and autonomous prescribing.	Approval blocked by mandatory tool/data/scope rules. Reviewer may reject or request a redesigned submission.

Journey 1 Complete review

Open GK-001, run the assessment, inspect the activity trace and open the Decision tab. Switch to Governance Reviewer, verify the oversight record, attest AI Governance review, enter a rationale and approve. Export the decision packet. The recorded approval applies only to the synthetic case; it does not trigger deployment.

Journey 2 Evidence collection

Run GK-002. Inspect its missing artifacts in the Evidence tab. Add synthetic sample evidence or paste a fictional evidence excerpt with metadata. Resume the same case and confirm that the review route includes Privacy and Security or Vendor Risk. Evidence attachment alone never approves the submission.

Journey 3 Failure recovery

Run GK-005 and inspect the two failed lookup attempts. Select Retry lookup. The same case resumes and retains the earlier errors in its trace. The outage is a deterministic test fixture, not an external vendor incident.

Tools data and model behavior

Tool	Inputs	Output and constraints
inspect_evidence	Evidence records and system version	Returns evidence kinds with required metadata and matching version. Does not establish truth or clinical validity.
lookup_tool	Tool ID; demo outage flag	Returns approved scope from the synthetic tools register. The orchestrator bounds failure attempts.
retrieve_policy	List of policy IDs	Returns exact catalog clauses with source versions. No external web retrieval occurs during assessment.

Three specialist roles

Intake and Evidence determines missing artifacts from structured case attributes. In optional live mode, its LangChain agent can inspect evidence and retrieve policy for advisory analysis. **Risk and Controls** checks approved tooling, maps risk rules and selects review routes; its optional agent can call policy and register tools. **Review Preparation** creates the human-facing packet and can use a model-generated advisory narrative. Specialist roles are sequential workflow nodes, not autonomous background workers.

Demo and live execution modes

Default mode is deterministic and requires no credentials. The real LangGraph workflow still calls real local LangChain tools. Setting GATEKEEPER_MODE=live with an OpenAI key enables three create_agent loops using a configurable model, defaulting to gpt-4.1-mini. The prompt asks for at most three read-only calls; a graph recursion limit of eight bounds execution. Provider calls use a 20-second timeout and one retry. A failure generates an explicit fallback event. Live provider execution has not been verified without credentials.

Data records

The catalog contains five fictional policy documents, four tool records and twelve controls. Six cases are seeded. Evidence includes an identifier, kind, title, author, date, system version, synthetic marker and content. Custom evidence is text-based in this MVP; binary PDF/DOCX ingestion is not implemented. A case revision number protects updates from stale browser state.

Grounding and source retention

Findings carry policy IDs, and exported packets include the catalog version and relevant policy records. Control mappings refer to the same catalog. Local traces show tool results and workflow actions without presenting private model reasoning. Optional LangSmith traces may contain synthetic case payloads when enabled.

GOVERNANCE DESIGN

Controls and human authority

Condition	Evidence or review requirement	Approval behavior
Any case	Named owner, oversight plan and AI Governance review	Reviewer must verify required evidence and attest required reviews.
PHI	Data flow and Privacy review; external processing also requires BAA evidence	Metadata checks do not constitute legal verification.
Clinical use	Clinical validation and Clinical Safety review	Human sign-off required.
Patient-facing use	Evaluation and monitoring plans	Human oversight must be validated.
Claimed de-identification	Documented method and Privacy review	Wrong-version evidence does not satisfy completeness.
Vendor conflict	Reconciliation evidence and Vendor Risk review	Reviewer must examine reported contradictions.
Unapproved tool or incompatible data	Mandatory blocker	Approval endpoint rejects the decision.
Autonomous prescribing	Outside permitted demonstration scope	Approval endpoint rejects the decision.

Decision gate

The API permits approval only from the Reviewer demo persona, in awaiting_review state, without blockers, with all required evidence IDs verified, all required review routes attested and a meaningful rationale. Request changes returns the case to the evidence loop. Finalized evidence is locked. Stale revision numbers are rejected.

Local demonstration limits

The persona header can be selected by any local user and is not identity security. Specialist sign-offs are attestations by one demo reviewer, not independent approvals. Case records and checkpoints are plaintext local SQLite databases; audit rows are append-only through the application, not cryptographically tamper-proof. These boundaries are visible in the app and must be addressed before exposure to real healthcare data or external users.

Validation and evaluation

The automated suite completed with 12 passing tests. The Next.js production build passed, including TypeScript validation. Browser checks exercised the staff approval, patient messaging evidence/resume, and discharge lookup failure/recovery journeys. These checks establish local workflow behavior, not clinical safety or regulatory compliance.

Test	Verified behavior
Complete case and human gate	Low-risk review, requester denial, missing verification denial and valid reviewer approval.
Missing evidence	Pause, re-pause when still incomplete, attach evidence and resume.
Conflict and changes loop	Reconciliation finding, request changes, revise evidence and reassess.
Evidence version	Previous-version evidence fails completeness.
Failure and persistence	Two failed attempts, checkpoint reopening with a new connection, successful retry and preserved errors.
Mandatory blocker	Complete evidence cannot override prohibited scope.
Auditor restrictions	Read-only persona cannot submit decisions or evidence.
Stale updates	Incorrect revision receives HTTP 409.
Export and citations	Decision packet contains audit records and valid policy references.
New intake validation	Custom case creation and persona restriction.
Live mode fallback	Missing key and simulated provider timeout preserve deterministic behavior without leaking credentials.

Recommended next evaluation set

Expand to at least twenty labeled cases with incomplete declarations, misleading document content, conflicting scope, unsupported data classes, repeated retries and interrupted writes. Measure review-route accuracy, missed mandatory gates, evidence grounding, successful recovery, completion time and reviewer usability. No aggregate model accuracy or time-saving claims have been established in this milestone.

Unverified integrations

Live OpenAI model responses and LangSmith delivery require configured credentials and an explicit end-to-end run. External deployment has not been performed. Responsive CSS is implemented; narrow-screen layout constraints were corrected; final narrow-screen browser verification remains incomplete.

Local operation and deployment path

Start and validate

From the repository root, run **./setup.sh**, then **./run-local.sh**. The setup creates a Python 3.12 environment and installs locked backend and frontend packages. The launcher checks ports 8100 and 3100 and starts both services on loopback. Open **http://127.0.0.1:3100**. API documentation is available at **http://127.0.0.1:8100/docs**.

Run **.venv/bin/python -m pytest -q tests** for workflow validation and **npm --prefix frontend run build** for the frontend production build. Tests use an isolated temporary store and do not modify the user demonstration cases. Local data lives in runtime/, which is excluded from Git.

Enable live agents and tracing

Copy .env.example to .env. Configure OPENAI_API_KEY and an available OPENAI_MODEL, and set GATEKEEPER_MODE=live. To enable LangSmith, configure LANGSMITH_API_KEY, LANGSMITH_PROJECT and LANGSMITH_TRACING=true. Restart the backend. Never commit .env. Confirm provider success in the activity trace and in the configured LangSmith project; configuration alone does not prove successful delivery.

Deployment after validation

The user selected local validation before deployment. The current milestone therefore has no hosted application URL. A later deployment can place Next.js and FastAPI on suitable services with managed persistent storage. Before publishing, replace persona switching with authenticated server-side roles, implement individually attributable review sign-offs, move to an encrypted managed database, add case-level access control, secure evidence ingestion and retention, and reconcile graph/audit writes across failures.

Five minute demonstration outline

Time	Demonstration
0:00 to 0:40	Product purpose, synthetic healthcare scope and personas.
0:40 to 1:40	Complete staff policy case, tool trace and human approval.
1:40 to 2:50	Patient messaging evidence interruption and resume.
2:50 to 3:50	Discharge case failure attempts and checkpoint recovery.
3:50 to 4:40	Policy citations, audit records and decision export.
4:40 to 5:00	Optional live agents, test evidence and production boundaries.

Companion videos include a narrated product and architecture overview and an edited walkthrough made from actual local app captures. The walkthrough is not an uninterrupted screen recording and does not claim to show live provider calls.

SUBMISSION CONTEXT

Build record and references

Alignment with the Week 3 handout

The October 2026 handout asks for an agentic system with tool use, persistent state, error recovery and human handoff. This implementation demonstrates those behaviors in a controlled local workflow. The optional live specialist mode supplies bounded model-selected tool use. The handout also asks for project documentation, code assets and a demo of five minutes or less. The source handout itself is not redistributed in the repository.

Implementation instructions and iterations

The user selected AI Gatekeeper, then specified a standalone healthcare app, a working local demo before deployment and the proposed technology and design defaults. Subsequent requests added a solution document, a product/architecture overview video, a live app walkthrough and a GitHub project. Implementation proceeded through a synthetic catalog and six cases, a persistent LangGraph backend, a Next.js review interface, automated workflow checks and browser validation. A later iteration added optional bounded live specialist agents, preserved reviewer verification in the finalized view and tightened narrow-screen layout constraints.

Model instruction used in the application

The specialist system prompt identifies the role, limits tools to read-only operations, treats supplied text as untrusted data, prohibits approval or record modification, prohibits invented facts or legal compliance claims, preserves deterministic gates and requests a concise reviewer-facing advisory summary. The exact prompt is version-controlled in backend/workflow.py. This prompt is a model instruction; authorization and policy gates are enforced separately in code.

Public background sources

HHS HIPAA and Cloud Computing

HHS De identification Guidance

NIST AI Risk Management Framework

LangGraph Interrupts

LangGraph Persistence

Next.js Installation

These sources provide background and implementation guidance. Synthetic internal rules are deliberately identified as such. They are not a comprehensive mapping of healthcare legal obligations.

Files for review

README.md explains setup, scope, mode boundaries and the demo journeys. backend/workflow.py contains the graph and tools. backend/main.py contains action validation and review gates. data/catalog.json contains the versioned synthetic sources. tests/test_workflow.py contains executable checks. The docs directory holds this solution document, diagrams, videos and narration transcripts.